

Inspeção automatizada de grãos de soja por visão computacional

Vígil.ia

Documento de contexto e transferência

Tudo que existe, por que existe, o que já foi descartado e o que falta fazer

*Este documento foi escrito para uma equipe que vai **assumir o projeto**. Ele privilegia o porquê das decisões — inclusive as que deram errado — porque o repositório acumula quatro gerações de código e boa parte dele é histórico, não código vivo.*

FATEC · Projeto Integrador · Agosto de 2026

Sumário

1. Leia isto primeiro — como o repositório está organizado
2. O que o projeto faz
3. As quatro eras do projeto (e por que isso importa)
4. Arquitetura: as peças e onde cada uma roda
5. Histórico de modelos: o que foi testado e o que venceu
6. O pipeline de dados (como o dataset é construído)
7. As regras de inferência que fazem o sistema funcionar
8. RF-DETR no Jetson Orin Nano — a frente mais recente
9. Estado atual: o que está pronto
10. O que falta — backlog priorizado
11. Armadilhas conhecidas (leia antes de debugar)
12. Mapa do repositório
13. Como rodar cada peça
14. Limitações honestas e o que NÃO afirmar

1. Leia isto primeiro — como o repositório está organizado

O repositório tem **quatro gerações de código sobrepostas**. Arquivos que parecem contraditórios normalmente são de eras diferentes, não erros. Antes de mexer em qualquer coisa, saiba em qual camada você está.

Camada	Status	O que fazer com ela
Detecção multi-grão / Jetson (jetson/ model/treino_rfdetr_*)	VIVO — frente atual	É aqui que o trabalho acontece hoje.
App local Steam Deck (deck/)	VIVO — funcional	Referência da lógica de voto/veredito. O app do Jetson espelha ele.
Web: foto + 1 grão (frontend/, backend/)	EM PRODUÇÃO — estável	Está no ar e funciona. Não é o foco atual, mas não quebre.
EfficientNet / Gradio (model/train.ipynb, gerar_relatorio.py)	HISTÓRICO	Só leitura. Não usar como referência técnica.
Docs antigos (ROADMAP.md, docs/ESPECIFICACAO.md, README.md)	DESATUALIZADO	Falam de soja_boa, modelos "Sinnet/Magnus", esteira/NIR. Não reflete o projeto atual.

Os documentos que valem

CLAUDE.md (contexto de desenvolvimento), CONTEXTO_PROJETO.md (estado consolidado) e model/COMPARATIVO_YOLO11S_VS_RTDETR.md (todos os experimentos de modelo, com os resultados negativos). Este PDF resume os três.

2. O que o projeto faz

Classificar grãos de soja por imagem, separando grão bom de grão com defeito. Existem **duas frentes de produto**, nascidas em momentos diferentes e que coexistem:

- **Modo foto (1 grão)** — o usuário fotografa um grão e recebe a classe. Está no ar como aplicação web.
- **Modo vídeo (multi-grão)** — vários grãos no mesmo quadro, ao vivo, com caixa e classe por grão. É a frente atual, mirando inspeção de lote.

As cinco classes

Índice	Rótulo (código)	Rótulo (interface)	Observação
0	broken	Quebrado	
1	immature	Imaturo	classe mais fraca hoje — ver §10
2	intact	Intacto	única classe "Premium"
3	skin-damaged	Casca danificada	fronteira ambígua com broken
4	spotted	Manchado	segunda classe mais fraca

A decisão que o sistema realmente toma

Apesar das cinco classes, a decisão de negócio é **binária**: intact → **Premium**; qualquer outra → **expulso da linha**. Isso importa na hora de avaliar o modelo — confundir spotted com skin-damaged não muda a decisão; confundir qualquer defeito com intact muda.

O custo é **assimétrico**: deixar passar um grão ruim é pior que descartar um grão bom. A regra de voto (§7) foi calibrada com isso em mente.

3. As quatro eras do projeto (e por que isso importa)

Entender a sequência evita reabrir decisões já resolvidas e explica por que há código aparentemente redundante.

Era	Stack	Como terminou
1. EfficientNet + Gradio	TensorFlow/Keras, modelo . keras, app Gradio	~29% de acurácia em fotos reais. Domain shift severo — o modelo só respondia "Quebrado". Arquivada.
2. YOLO11s-cls + web	Ultralytics/PyTorch, FastAPI, Next.js, Supabase	91,7% após fine-tuning no domínio real. Está em produção.
3. Detecção em vídeo	YOLO11 n/s/l/x e RT-DETR, cenas sintéticas, rastreamento	YOLO11x virou campeão de vídeo. Roda em GPU de servidor.
4. RF-DETR para edge	RF-DETR Small, TensorRT, Jetson Orin Nano	Validado: 53 qps no Jetson. Frente atual.

A lição que atravessa as quatro eras: *domain shift*

O gargalo nunca foi a arquitetura — foi a diferença entre as imagens de treino e as de uso real. Na era 1 isso custou ~60 pontos de acurácia (29% vs ~85% no dataset de laboratório). Toda vez que a câmera, a luz ou o fundo mudam, o modelo precisa de dado novo. **Guarde isso: é a explicação de quase todo resultado ruim deste projeto.**

4. Arquitetura: as peças e onde cada uma roda

São quatro formas independentes de rodar inferência. Elas compartilham as classes e a lógica de veredito, mas têm modelos e ambientes diferentes.

Peça	Onde roda	Modelo	Para quê
Web — modo foto	Hugging Face Spaces (Docker) + Vercel	YOLO11s-cls	Produção. Foto de 1 grão → classe + análise por IA.
Web — modo ao vivo	Navegador do usuário (ONNX Runtime Web)	YOLO11s-cls (ONNX)	Demo instantânea, sem servidor.
App Steam Deck	Local, 100% offline	YOLO11n	Inspeção ao vivo sem internet. Referência da lógica de voto.
Servidor Colab	Colab (GPU) + túnel Cloudflare	YOLO11x / RT-DETR	Demo dos modelos pesados com câmera de celular.
Jetson Orin Nano	Embarcado, TensorRT	RF-DETR Small FT4	Frente atual — caminho para o produto.

Fluxo do modo web (produção)

foto → POST /inspect (FastAPI no HF Spaces) → YOLO11s-cls classifica
→ grava no Supabase → tela de resultado
→ (opcional) usuário pergunta → /api/explain → Groq · Llama 3.3

Fluxo do modo vídeo (Jetson, frente atual)

câmera → letterbox 512×512 → engine TensorRT (RF-DETR)
→ NMS agnóstico de classe → rastreamento por IoU
→ voto ponderado por grão → veredito travado → desenho na tela

O celular entra como câmera via DroidCam (URL de rede) ou modo webcam USB. O Jetson também tem conectores CSI, mas trocar de câmera introduz domain shift novo — ver §11.

5. Histórico de modelos: o que foi testado e o que venceu

Todos os comparativos seguiram o mesmo método: **mesmo dado, mesmo pipeline, o experimento decide**. Vários resultados contrariaram a intuição inicial.

5.1 Classificação (1 grão) — decidido

Modelo	Acurácia (domínio real)	Gap treino-val
EfficientNet-B0 — receita antiga	64,0%	25,0% (overfitting)
EfficientNet-B0 — em igualdade de condições	75,0%	7,2%
YOLO11s-cls — em produção	91,7%	3,9%

Ressalva: essa validação usou apenas 12 fotos. 91,7% = 11 de 12. É tendência forte, não métrica estatística.

5.2 Detecção em vídeo — a hipótese inicial estava errada

A aposta era que o RT-DETR (transformer) venceria a família YOLO. Depois de corrigir uma assimetria metodológica — os YOLOs não estavam recebendo o mesmo estágio base de 12,5 mil imagens — o resultado se inverteu:

Modelo	Parâmetros	GFLOPs	Acurácia premium (visual)
YOLO11s (sem estágio base)	9,4 M	21,5	~65-75%
YOLO11l	25,3 M	86,9	≈ RT-DETR
RT-DETR-l	32,0 M	103,5	~80-85%
YOLO11x	56,9 M	194,9	~95% — campeão

Conclusão: o gap era **capacidade + caminho de dados**, não arquitetura. A tese "transformer generaliza melhor com dado escasso" vale apenas no regime de capacidade igual ou menor.

5.3 RF-DETR para o Jetson — quatro tentativas de fine-tuning

O YOLO11x é modelo de servidor, inviável no Orin Nano. O RF-DETR Small foi testado como candidato a edge, pelo mesmo caminho de estágios. As quatro variações de fine-tuning contam uma história útil:

Rodada	O que foi treinado	Resultado
Estágio 1	Base 12,5k, pseudo-rótulo Otsu	mAP 0,985 — saturou rápido (tarefa fácil demais)
FT1	Fotos reais + vídeo de defeito + cenas sintéticas	mAP 0,826, todas as classes saudáveis
FT2	Só as capturas do vigil_deck	Piorou — mAP 0,375, esquecimento catastrófico. Perdeu até para o FT1 sozinho.
FT3	Capturas + 30% de replay do FT1	Resolveu o esquecimento. Melhor que FT1 e FT2.
FT4	Fonte do recorte escolhida por classe	Campeão. Parou de confundir imaturo com intacto.

O achado mais útil do FT4 (e não é o que parecia)

A hipótese ao montar o FT4 era de **qualidade de imagem** — cada fonte teria fotos melhores de classes diferentes. O que aconteceu foi outra coisa: as fotos de immature do FT1 estavam **mal rotuladas** e ensinavam que "imaturo se parece com intacto". Trocar a fonte dessa classe não deu imagens melhores — **removeu o contra-exemplo errado**.

Ou seja, o mecanismo funcionou como **filtro de qualidade de rótulo**. É um contorno, não uma correção: o dado ruim continua na pasta de origem.

6. O pipeline de dados (como o dataset é construído)

Esta é a parte mais importante para quem for treinar. Quase todo ganho do projeto veio de mexer **aqui**, não em hiperparâmetro nem em arquitetura.

Fontes

Fonte	O que é	Papel
Dataset Roboflow	12.528 imagens 400×400, licença MIT, 1 grão por foto	Estágio base. Vira detecção via pseudo-rótulo Otsu (fundo preto → caixa automática, sem anotação manual).
Fotos reais	Fotos do celular, 1 grão, fundo controlado	Fine-tuning no domínio real. Sem isso a acurácia despenca.
Capturas	Recortes salvos pelo <code>vigil_deck --save-dir</code>	Único dado vindo do setup de uso real. Recortes apertados.
Vídeos de defeito	Frames de vídeo por classe	Enriquece o FT1. <code>intact</code> é excluído de propósito para manter a avaliação limpa.

Cenas sintéticas multi-grão — o truque central

Fotos de 1 grão não ensinam um detector a lidar com quadro cheio. A solução foi gerar cenas artificiais:

- Recortar cada grão da foto real com máscara por saturação, **erodida em 1px** para eliminar o halo que inflava a caixa.
- Colar de 6 a 25 grãos num fundo escuro gerado, com rotação e sobreposição controlada, borda nítida (feather pequeno).
- Aplicar *motion blur* em ~40% das cenas, simulando movimento de esteira.
- Balancear o **pool de recortes** por classe — o gerador sorteia uniforme, então pool torto produz cena torta.

Resultado negativo importante: NÃO balanceie o treino por classe

Balancear o *train* (e não só o *val*) deixou o modelo viciado em defeito. O vídeo real é majoritariamente `intact`, e treinar com 20% de cada classe ensina que defeito é comum. **O val deve ser balanceado; o train deve espelhar a realidade.**

Proporção final do dataset campeão (FT4)

Origem	Imagens	Caixas	% das caixas
Fotos soltas (1 grão)	1.892	1.892	9%
Cenas sintéticas (6-25 grãos)	1.200	18.477	91%

Essa proporção explica um comportamento que parece defeito e não é: o modelo vai **muito bem em multi-grão e mal em grão solto**. Com 91% do treino em cena densa, e considerando que a família DETR aprende um *prior de quantos objetos existem*, um grão sozinho está fora da distribuição. Está alinhado com o uso real.

7. As regras de inferência que fazem o sistema funcionar

O modelo sozinho oscila entre quadros. O que torna o sistema utilizável são três regras aplicadas **por cima** da detecção. Elas estão em `deck/vigil_deck.py` e replicadas no app do Jetson.

1. Voto exigente por classe

Cada grão acumula votos ponderados pela confiança ao longo dos quadros. Um defeito só é aceito se atingir uma fração mínima dos votos; senão o grão recebe o benefício da dúvida e vira `intact`.

Classe	Fração mínima de votos	Leitura
broken	0,85	mais exigente — era a que mais alucinava
skin-damaged	0,80	
spotted	0,75	
immature	0,75	
intact	—	ganha direto (é o padrão de segurança)

2. Veredito travado por grão

A classe de um grão só é fixada depois de **8 quadros** rastreando e **3 segundos** de observação. Antes disso ele aparece como "analizando...". Isso mata a troca de rótulo a cada quadro.

3. Caixa suavizada e anti-piscada

A caixa é suavizada por média móvel (fator 0,4) para não tremer, e detecções que aparecem em menos de 3 quadros são descartadas como ruído.

Esses números foram calibrados observando vídeo real. Se o setup de captura mudar, eles precisam ser recalibrados — mas só depois da padronização (§10).

8. RF-DETR no Jetson Orin Nano — a frente mais recente

Objetivo: sair da GPU de servidor e rodar embarcado. O RF-DETR é da família DETR, com backbone DINOv2 e arquitetura definida por busca automática (NAS).

Por que RF-DETR Small

As variantes Nano a Large têm **quase o mesmo número de parâmetros** (~30-34 M) — o que muda entre elas é principalmente a **resolução de entrada**. O Small (512px) foi escolhido como meio-termo: mais detalhe que o Nano (384px), o que importa em objeto pequeno como grão.

Resultado medido no aparelho

Métrica	Valor
Throughput	53,2 qps (TensorRT FP16, modo 15W)
GPU Compute (média)	18,68 ms
Engine em disco	59 MB
Veredito	Cabe em tempo real, com folga

Não foi preciso INT8, nem cair para o RF-DETR Nano, nem usar DeepStream. **Nota de método:** a estimativa prévia era 35-40 qps, extrapolada de um AGX Orin pela razão de TOPS. O medido veio acima — escalar performance por TOPS subestima.

Como o modelo chega ao Jetson

.pth (Colab) → treina e avalia; exige rfdetr + torch
.onnx (ponte) → formato neutro, exportado NO COLAB
.engine → gerado NO PRÓPRIO JETSON, atado ao aparelho e à versão do TensorRT

A engine não é portátil

Ela é compilada para este hardware e esta versão de TensorRT. Não adianta gerar no PC e copiar. Se o JetPack for atualizado, reconstrua.

9. Estado atual: o que está pronto

Item	Status
Web modo foto (produção)	■ No ar — HF Spaces + Vercel + Supabase
Web modo ao vivo (ONNX no navegador)	■ Funcional
App Steam Deck (offline)	■ Funcional, com captura para treino
Servidor de demo no Colab + túnel	■ Funcional
Modelo de vídeo em GPU (YOLO11x)	■ Campeão validado
RF-DETR Small no Jetson	■ 53 qps medidos, app ao vivo rodando
Pipeline de treino reproduzível	■ Notebook único, resumível por estágio
Documentação dos experimentos	■ Inclui os resultados negativos

10. O que falta — backlog priorizado

Prioridade 1 — e ela bloqueia quase tudo: padronizar a captura

O modelo foi treinado com fotos do celular. O rig do Jetson tem outra câmera, outra luz e outra distância — ou seja, **outro domínio**. Qualquer número de acurácia medido agora não se transfere.

Enquanto não houver padrão, meça só o que independe de domínio: fps, latência, estabilidade de caixa, integração. **Acurácia, recall por classe e calibragem dos limiares ficam suspensos.**

O padrão precisa fixar: distância, fundo, enquadramento, câmera e **iluminação travada** — sem exposição nem balanço de branco automáticos variando entre sessões, que é o jeito mais fácil de introduzir domain shift sem perceber.

#	Tarefa	Por quê / observação
1	Definir o padrão de inspeção e captura	Bloqueia todo o resto. Sem isso, dado e métrica nascem descartáveis.
2	Coletar ~120 grãos únicos de immature e spotted	Só depois do item 1. Hoje há 56 e 35. São as duas classes fracas. Decidido: capturar grão novo, não corrigir rótulo (o erro no FT1 foi sistemático — as fotos não são de grãos imaturos).
3	Gravar vídeo de lote propositalmente ruim	O vídeo de teste atual é quase todo intact, o que não distingue "modelo bom" de "modelo viciado em dizer intacto".
4	Re-treinar no domínio padronizado	Com 1-3 prontos, esta vira a rodada de fine-tuning que realmente conta.
5	Recalibrar limiares de voto e confiança	Os valores atuais foram calibrados noutro domínio.
6	Testar RF-DETR Nano (em andamento)	Hipótese: modelo menor sofre menos overfit com dado escasso. Contra-pressão: 384px pode perder detalhe em objeto pequeno.

11. Armadilhas conhecidas (leia antes de debugar)

Esta seção existe porque alguns erros se repetiram várias vezes. Reconhecer o padrão economiza horas.

Deslocamento de índice de classe — aconteceu três vezes

O modelo devolve índices numéricos; o código traduz para nomes. Toda vez que a convenção de indexação foi **presumida** em vez de verificada, os rótulos saíram deslocados em 1 — e o sistema continuou funcionando, só que mentindo.

Sintoma observado	Causa real
Vídeo mostrava <code>immature</code> dominando, e <code>spotted</code> nunca aparecia	Índice do modelo era 0-based; o código mapeava por ID do COCO (1-based)
App do Jetson dizia <code>immature</code> para tudo	A cabeça exporta 6 colunas para 5 classes; assumiu-se que a coluna extra era fundo na frente , mas o DETR põe o "no-object" no fim

Como detectar: se uma classe some completamente do resultado, ou se uma classe rara domina, suspeite de deslocamento antes de suspeitar do modelo. O `jetson/vigil_jetson.py --diag` imprime um histograma das colunas cruas e resolve a dúvida com dado.

Falha silenciosa em biblioteca

Duas vezes uma dependência mudou de comportamento sem erro: o rastreador ByteTrack do supervision foi depreciado e passou a devolver identificadores vazios — o modelo detectava normalmente, mas **nenhum voto era contado**. E o `cuda-python 13` reorganizou os módulos, quebrando o `import`.

Lição aplicada: o app do Jetson usa um rastreador próprio de ~40 linhas em vez de depender de biblioteca externa, e checa códigos de retorno da GPU. Onde há falha silenciosa possível, prefira falhar alto.

Métrica de validação que mede outra coisa

Numa rodada, o mAP caiu de 0,92 para 0,43 e parecia colapso do modelo. Não era: os dois estágios validavam em conjuntos **diferentes**, com distribuições de classe diferentes. No vídeo real o modelo "pior" era melhor.

Regra prática: só compare números medidos na mesma régua. Entre estágios e entre versões, **o vídeo real é o único termômetro comparável**.

Benchmark em modo de energia reduzido

O Jetson inicia em modo econômico (15W). Medir sem ajustar isso dá número artificialmente baixo. Igualmente importante: **não misture modos** ao comparar modelos — número de 15W não se compara com número de MAXN.

Variação de ~15% entre execuções do mesmo modelo é normal neste setup (estado térmico, clocks). Só diferença grande é sinal.

12. Mapa do repositório

jetson/	FRENTE ATUAL – embarcado
vigil_jetson.py	app ao vivo (TensorRT + voto + veredito)
bench_trt.sh	constrói a engine e mede
inspect_engine.py	mostra entradas/saídas da engine
comparar_engines.py	compara modelos no mesmo vídeo
check_espaco.sh	diagnóstico de armazenamento
README.md	guia completo do fluxo
deck/	app local Steam Deck (offline)
vigil_deck.py	REFERÊNCIA da lógica de voto/veredito
model/	treino e experimentos
treino_rfdetr_small_completo.ipynb	pipeline RF-DETR (nano small)
COMPARATIVO_YOLO11S_VS_RTDETR.md	TODOS os experimentos
PROTOCOLO_ANOTACAO_VIDEO.md	protocolo de captura
aprendizado_ativo.ipynb	correção humana → re-treino
testar_video.py	caixas cruas de um modelo
demo_servidor_colab.ipynb	servidor GPU + túnel
train.ipynb, finetune.ipynb	(histórico – EfficientNet)
backend/	FastAPI de produção (modo foto)
frontend/	Next.js (web + modo ao vivo ONNX)
CLAUDE.md	contexto de desenvolvimento
CONTEXTO_PROJETO.md	estado consolidado
ROADMAP.md, docs/	DESATUALIZADOS

13. Como rodar cada peça

Treinar (Google Colab, GPU)

```
Abrir model/treino_rfdetr_small_completo.ipynb
Ajustar VARIANTE = "nano" ou "small"
Executar de cima para baixo. Cada estágio salva no Drive e pula
se já existir — dá para dividir em várias sessões.
```

Levar o modelo ao Jetson

```
# no Colab: rodar a célula "Export ONNX"
# copiar para o Jetson:
scp soja_rfdetr_*_CAMPEAO.onnx bench_trt.sh usuario@IP:~/

# no Jetson: construir a engine e medir
chmod +x bench_trt.sh
./bench_trt.sh soja_rfdetr_small_CAMPEAO.onnx fp16
```

Rodar a inspeção ao vivo no Jetson

```
sudo apt install -y python3-pip python3-dev
pip3 install cuda-python --break-system-packages

python3 vigil_jetson.py                # celular (DroidCam)
python3 vigil_jetson.py --camera 0      # webcam USB
python3 vigil_jetson.py --camera csi    # câmera CSI
python3 vigil_jetson.py --source video.mp4 --out saida.mp4
python3 vigil_jetson.py --diag 100     # diagnóstico de classes

# teclas: q sai · c zera contagem · p pausa
```

Rodar o app do Steam Deck

```
python3 -m venv ~/vigilia && source ~/vigilia/bin/activate
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install ultralytics opencv-python
python vigil_deck.py --camera http://IP_DO_CELULAR:4747/video
python vigil_deck.py --save-dir capturas # coleta para treino
```

14. Limitações honestas e o que NÃO afirmar

Esta seção existe para evitar que números sejam repetidos fora de contexto — inclusive em apresentação ou relatório.

O que se pode dizer	O que NÃO se pode dizer
O RF-DETR Small roda a 53 qps no Orin Nano (medido, TensorRT FP16, 15W)	Que o sistema "tem 95% de acurácia" — os números de vídeo são avaliação visual , não benchmark rotulado
O modelo vai bem em cena multi-grão no vídeo de teste	Que ele funciona em qualquer lote — o vídeo de teste é quase todo intact
Três das cinco classes estão sólidas no val (broken 0,86 · intact 0,81 · skin-damaged 0,67)	Que as cinco classes funcionam — immature tem recall 0,041 e spotted AP 0,30
O modo foto atingiu 91,7% no domínio real	Que isso é estatisticamente robusto — foram 12 fotos de validação

A limitação mais importante de todas

O modelo **não detecta grão imaturo** na prática (recall 4%). O vídeo de teste não contém nenhum grão imaturo, então isso não aparece na avaliação — "não confundir" e "não detectar" produzem a mesma imagem quando a classe está ausente. **Um lote com grão imaturo passaria batido.** Foi uma decisão consciente (é o melhor ponto para o dado disponível hoje), mas precisa ser revisitada antes de qualquer uso real.

Fontes: CONTEXTO_PROJETO.md, model/COMPARATIVO_YOLO11S_VS_RTDETR.md, jetson/README.md e o histórico de commits do repositório.